

Machine Learning

Regularization



Dr. G. Omprakash

KLEF Hyderabad (Aziz Nagar) – Dept. of ECE

Machine Learning (25SC2107E)

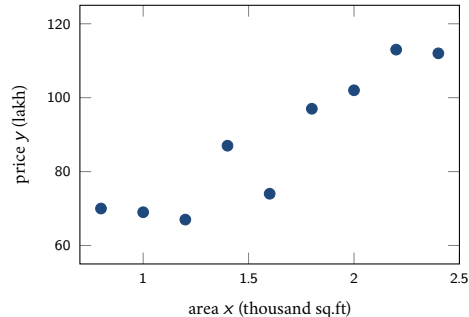
Session Outcome

- Understand why a model with many features fits the training data well but fails on unseen data, and control this using *Ridge*, *Lasso* and *Elastic Net* regularisation.

Regularization

The Dataset: Nine Houses, One Feature

House	Area $\times$ (k sq.ft)	Price y (lakh)
1	0.8	70
2	1.0	69
3	1.2	67
4	1.4	87
5	1.6	74
6	1.8	97
7	2.0	102
8	2.2	113
9	2.4	112



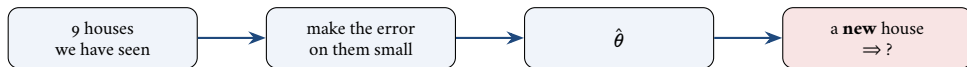
Observations

- Bigger flats cost more
- The trend is clearly rising and roughly straight.

Minimizing MSE

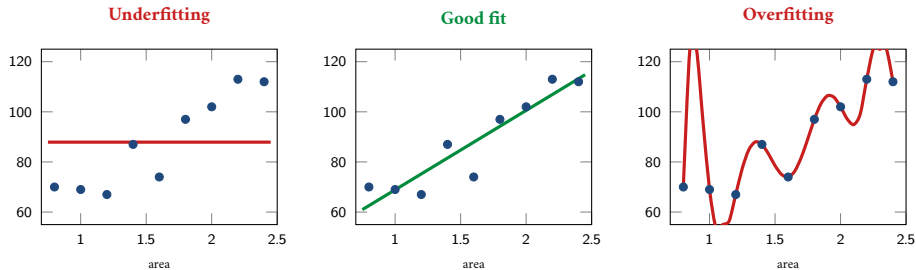
Every model so far was fitted by making the training error as small as possible:

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \left(\theta^\top \mathbf{x}^{(i)} - y^{(i)} \right)^2 \quad \Rightarrow \quad \hat{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$



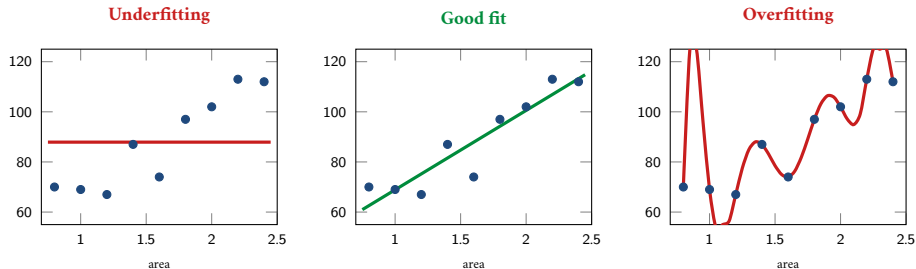
- No conditions in $J(\theta)$ to bound θ
- Is a smaller training error always a better model?
- Let us check with new data.

Three Models for the Same Dataset



For the same dataset we have three different models

Three Models for the Same Dataset



For the same dataset we have three different models

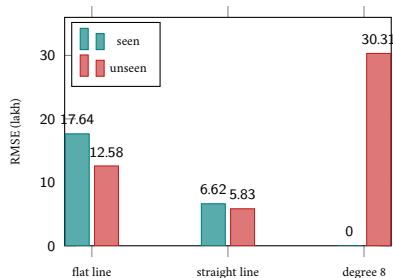
- **Underfitting:** one flat price for everybody!! (Simple, but misses the trend)
- **Good fit:** follows the trend.
- **Overfitting:** passes through *every* point exactly.
 - It even dives below near $x = 1.1$
 - Shoots above 125 lakh near $x = 0.85$

RMSE for Three Models

Model	RMSE on the 9 seen houses	RMSE on the 8 unseen
flat line (underfit)	17.64	12.58
straight line (good)	6.62	5.83
degree-8 curve (overfit)	0.00	30.31

Observation

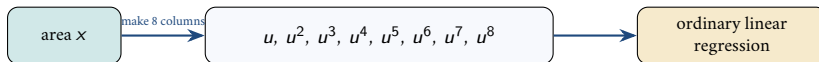
The overfitted model has the best training score with zero error and a perfect fit. It is also, by a wide margin, the worst model of the three.



Overfit Model: Still a Linear Regression

We have only one measurement, area x . To let the model bend, we use higher degree polynomial:

$$\hat{y} = \theta_0 + \theta_1 u + \theta_2 u^2 + \theta_3 u^3 + \cdots + \theta_8 u^8, \quad u = \frac{x - 1.6}{0.8}$$

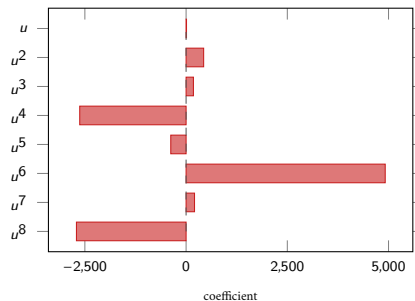


Why rescale to u ?

With x in thousands, x^8 would range over four orders of magnitude. Bring the range to $[-1, 1]$, so every column has a comparable size. *This is standardisation*

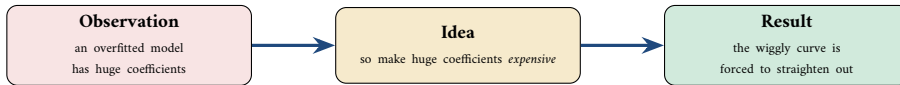
Look Inside the Overfitted Model

Term	Coefficient θ_i
θ_0 (intercept)	74.00
$\theta_1 u$	10.10
$\theta_2 u^2$	433.62
$\theta_3 u^3$	181.16
$\theta_4 u^4$	-2626.76
$\theta_5 u^5$	-378.31
$\theta_6 u^6$	+4918.04
$\theta_7 u^7$	208.05
$\theta_8 u^8$	-2707.91
$\sum_j \theta_j $	11 464



Overfitting is not a mysterious idea: it is visibly large numbers in θ .

Regularization



$$J(\theta) = \underbrace{\frac{1}{m} \|\mathbf{X}\theta - \mathbf{y}\|^2}_{\text{fit the 9 houses}} + \lambda \underbrace{\text{pen}(\theta)}_{\text{keep } \theta \text{ small}}$$

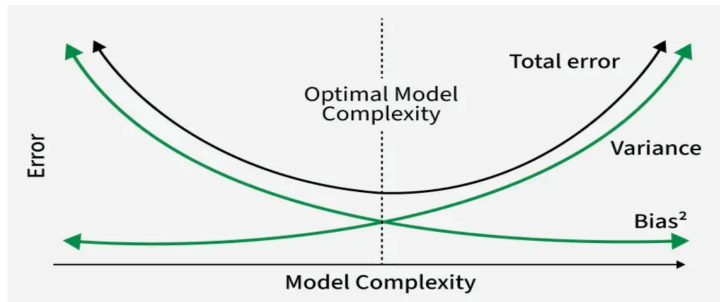
- Three choices of $\text{pen}(\theta)$ give three famous methods:

Ridge	$\lambda \sum_j \theta_j^2$	squares — shrinks everything smoothly
Lasso	$\lambda \sum_j \theta_j $	absolute values — deletes some terms outright
Elastic Net	mixture of both	shrinks <i>and</i> deletes

Bias and Variance

Bias and Variance are two types of prediction errors

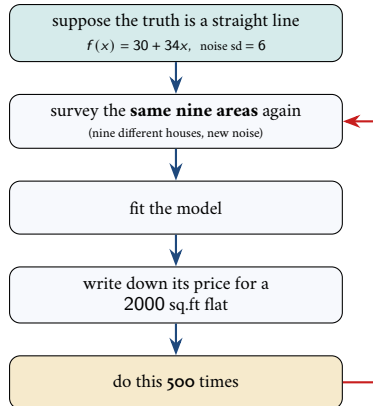
- **Bias** refers to the error caused by making wrong or simple assumptions
 - Leads to **underfitting**
- **Variance** refers to the error caused by making the model too sensitive to training data.
 - Leads to **overfitting**



Source: GeekforGeeks

Figure: Bias-Variance Tradeoff

Experiment to Reveal Bias and Variance



The two questions we can now answer

Bias: is the *average* of the 500 answers right?

Variance: how much do the 500 answers disagree with each other?

- True price of that flat: $f(2.0) = \mathbf{98.0}$ lakh.

Redraw the Fit

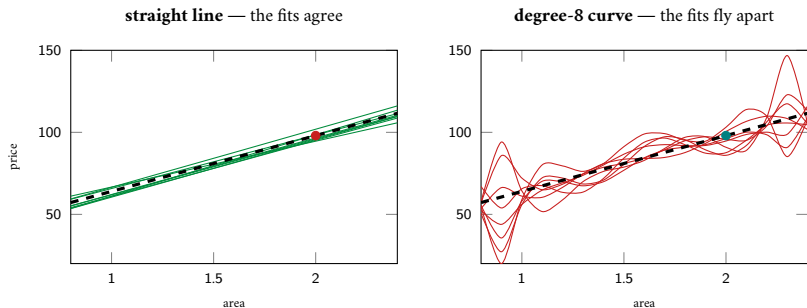
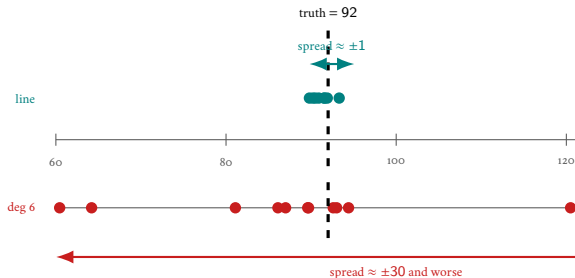


Figure: Dashed black = the truth. Each thin line is one repeat of the survey.

Left: change the houses, get almost the same line. Right: change the houses, get a completely different curve.

Picture 2: the Same Eight Fits Seen at One House



Measured at Area = 2.0 over 500 repeats

	line	deg 6
average prediction	91.51	93.06
truth	92.00	92.00
bias	-0.49	+1.06
Bias ²	0.24	1.13
Variance	2.07	229.16

$$\bar{y} = \frac{1}{S} \sum_{s=1}^S \hat{y}^{(s)}$$

$$\text{Bias} = \bar{y} - f(x^*)$$

$$\text{Bias}^2 = (\bar{y} - f(x^*))^2$$

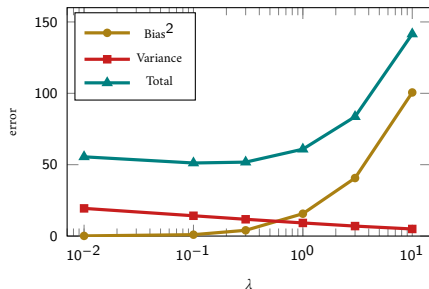
$$\text{Variance} = \frac{1}{S} \sum_{s=1}^S (\hat{y}^{(s)} - \bar{y})^2$$

The Trade, Measured: Penalising the Degree-8 Curve

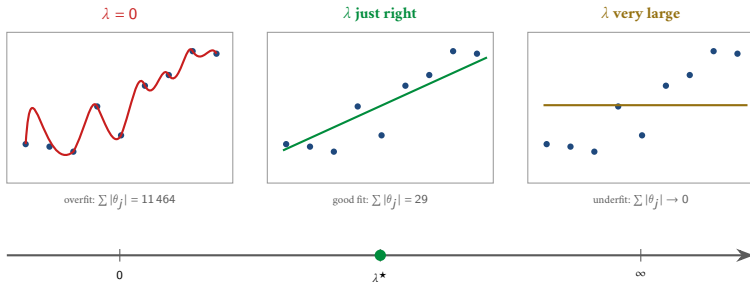
Total Error = Bias² + Variance + Noise

Same degree-8 model, same 500 surveys — but fitted with a penalty $\lambda \sum_j \theta_j^2$.

λ	Bias ²	Variance	Noise	Total
0	0.10	103.12	36	139.21
0.01	0.12	19.39	36	55.51
0.1	0.97	14.22	36	51.19
0.3	4.03	11.76	36	51.79
1	15.66	9.19	36	60.86
3	40.64	6.99	36	83.63
10	100.59	4.97	36	141.56



The Knob λ



- λ (called `alpha` in `scikit-learn`) is the **only** new knob.

Ridge Regression

$$J(\theta) = \|\mathbf{X}\theta - \mathbf{y}\|^2 + \lambda \sum_{j=1}^8 \theta_j^2$$

Derivation. Set the gradient to zero, using $\nabla_{\theta}(\theta^T \theta) = 2\theta$:

$$2\mathbf{X}^T(\mathbf{X}\theta - \mathbf{y}) + 2\lambda\theta = \mathbf{0} \implies (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})\theta = \mathbf{X}^T\mathbf{y} \implies \hat{\theta}_{\text{ridge}} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$$

Why the extra λI matters

Every eigenvalue $\mu_i \geq 0$ becomes $\mu_i + \lambda > 0$, so $\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}$ is **always invertible**. Ridge **repairs** the normal equation.

The name

A “ridge” of height λ is added along the diagonal:

$$\begin{pmatrix} \square + \lambda & \square & \square \\ \square & \square + \lambda & \square \\ \square & \square & \square + \lambda \end{pmatrix}$$

Ridge has a **closed form**, exactly like ordinary least squares — one line of linear algebra, no iteration.

Ridge by Hand — Our Nine Houses, Two Terms

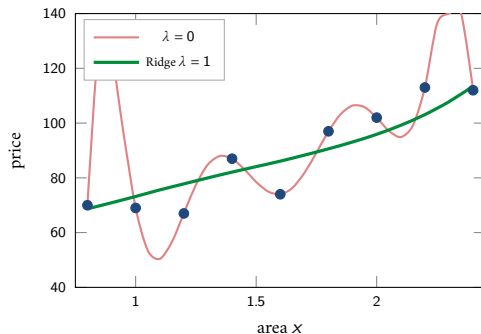
Take the same 9 houses but allow only u and u^2 , and centre the prices ($\bar{y} = 87.89$) so no intercept is needed. Because $u = -1, -0.75, \dots, 1$ is symmetric, the two columns are **orthogonal** and $\mathbf{X}^\top \mathbf{X}$ is diagonal:

$$\mathbf{X}^\top \mathbf{X} = \begin{pmatrix} 3.750 & 0 \\ 0 & 2.766 \end{pmatrix}, \quad \mathbf{X}^\top \mathbf{y} = \begin{pmatrix} 95.00 \\ 8.54 \end{pmatrix} \quad \Rightarrow \quad \hat{\theta}_j = \frac{(\mathbf{X}^\top \mathbf{y})_j}{d_j + \lambda}$$

$$\begin{aligned} \lambda = 0 : \quad \theta_1 &= \frac{95.00}{3.750} = 25.33, & \theta_2 &= \frac{8.54}{2.766} = 3.09 \\ \lambda = 1 : \quad \theta_1 &= \frac{95.00}{4.750} = 20.00, & \theta_2 &= \frac{8.54}{3.766} = 2.27 \\ \lambda = 3 : \quad \theta_1 &= \frac{95.00}{6.750} = 14.07, & \theta_2 &= \frac{8.54}{5.766} = 1.48 \\ \lambda = 10 : \quad \theta_1 &= \frac{95.00}{13.75} = 6.91, & \theta_2 &= \frac{8.54}{12.77} = 0.67 \end{aligned}$$

λ	θ_1	θ_2	$\ \theta\ ^2$	train SSE
0	25.33	3.09	651.3	367.8
1	20.00	2.27	405.2	476.4
3	14.07	1.48	200.3	850.4
10	6.91	0.67	48.2	1657.0
30	2.82	0.26	8.0	2291.5

Ridge on the Full Degree-8 Model

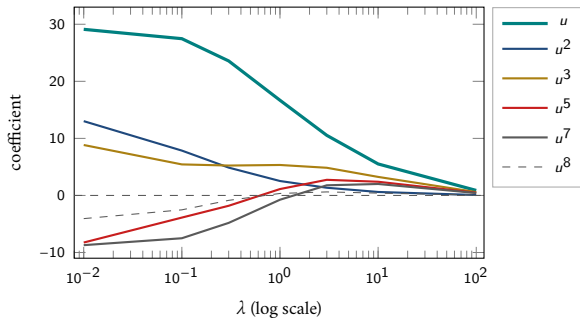


Term	$\lambda = 0$	Ridge $\lambda = 1$
θ_0	74.00	86.14
u	10.10	16.68
u^2	433.62	2.53
u^3	181.16	5.34
u^4	-2626.76	1.35
u^5	-378.31	1.12
u^6	4918.04	0.74
u^7	208.05	-0.75
u^8	-2707.91	0.35
$\sum_j \theta_j $	11 464	28.8
train RMSE	0.00	7.36
test RMSE	30.31	4.47

What the penalty fixed

All eight powers are **still there** — none was deleted — but every one has been squeezed to single digits. The curve straightens out on its own, and the error on unseen houses drops from 30.31 to **4.47**: better than the straight line we drew by hand.

Ridge: How the Coefficients Travel as λ Grows



- Every curve slides **smoothly** towards zero.
- Not one of them ever **reaches** zero — look at $\lambda = 100$: the smallest is still 0.06, not 0.
- So Ridge always hands you a model that uses **all eight** powers, however tiny their weights.

Ridge in four words

It shrinks, never deletes.

Lasso Regression

Ridge

$$J = \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2 + \lambda \sum_j \theta_j^2$$

$$\hat{\theta}_j^{\text{ridge}} = \frac{c_j}{d_j + \lambda},$$

$$\hat{\theta}_j^{\text{lasso}} = \frac{\text{sign}(c_j) \max(|c_j| - \frac{\lambda}{2}, 0)}{d_j},$$

Lasso

$$J = \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2 + \lambda \sum_j |\theta_j|$$

$$c_j = (\mathbf{X}^T \mathbf{y})_j$$

with $c_1 = 95.00$, $d_1 = 3.750$ and $c_2 = 8.54$, $d_2 = 2.766$.

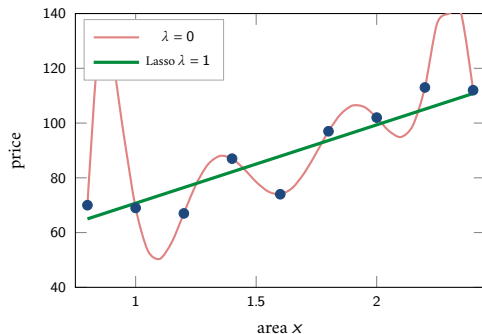
Ridge

λ	$\theta_1 (u)$	$\theta_2 (u^2)$
0	25.33	3.09
1	20.00	2.27
3	14.07	1.48
10	6.91	0.67
30	2.82	0.26

Lasso

λ	$\theta_1 (u)$	$\theta_2 (u^2)$
0	25.33	3.09
10	24.00	1.28
20	22.67	0.00
40	20.00	0.00
400	0.00	0.00

Lasso on the Full Degree-8 Model: the Picture

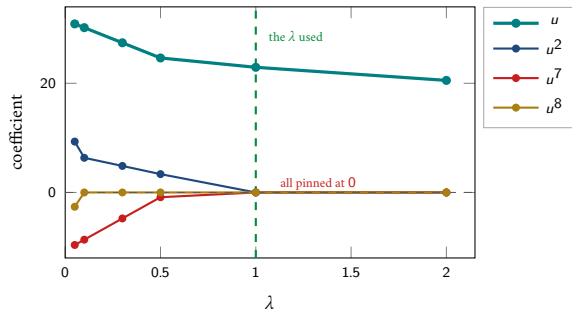


Term	$\lambda = 0$	Lasso $\lambda = 1$
θ_0	74.00	87.89
u	10.10	22.93
u^2	433.62	0
u^3	181.16	0
u^4	-2626.76	0
u^5	-378.31	0
u^6	4918.04	0
u^7	208.05	0
u^8	-2707.91	0
terms used	8	1
train RMSE	0.00	6.80
test RMSE	30.31	5.27

Lasso rediscovered the straight line

Given eight powers to play with, Lasso threw **seven of them away** and kept only u . The green curve is not a curve at all — it is a line. Nobody told the model that the market is linear; it worked that out from the data.

Lasso: the Powers Switch Off One by One



λ	alive	switched off here
0	8	—
0.05	4	u^3, u^4, u^5, u^6
0.1	3	u^8
0.5	3	—
1	1	u^2, u^7
4	1	—

- Once a curve touches zero it **stays** there.
- The weakest terms die first — so the death order is a **ranking of usefulness**.
- The last survivor is u : the one term the data really supports.

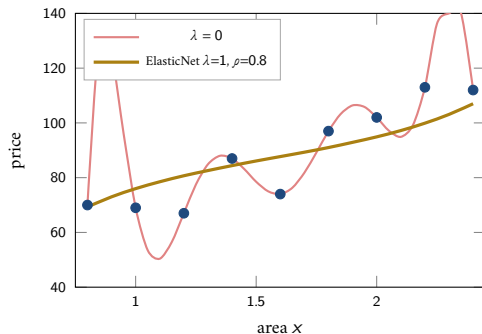
Elastic Net: Mix the Two Penalties

$$J(\theta) = \|\mathbf{X}\theta - \mathbf{y}\|^2 + \lambda \left[\underbrace{\rho \sum_j |\theta_j|}_{\text{deletes}} + (1 - \rho) \underbrace{\sum_j \theta_j^2}_{\text{steadies}} \right], \quad \rho \in [0, 1]$$

The corners survive, so zeros are still possible; the edges bulge, so correlated terms stay together.

- $\rho = 1$ gives pure Lasso; $\rho = 0$ gives pure Ridge.
- The L_1 part still produces **exact zeros**.
- The L_2 part makes the problem **strictly convex**, so the answer is unique — correlated terms are made to share rather than fight.
- Two knobs: λ (how hard) and ρ (which mixture). Tune both on validation data.

Elastic Net on the Full Degree-8 Model: the Picture

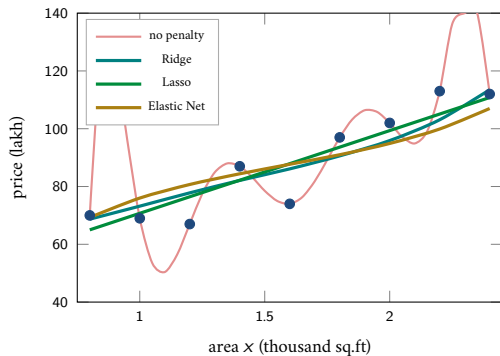


Term	$\lambda = 0$	ElasticNet
θ_0	74.00	87.70
u	10.10	12.97
u^2	433.62	0.44
u^3	181.16	4.54
u^4	-2626.76	0.02
u^5	-378.31	1.37
u^6	4918.04	0
u^7	208.05	0
u^8	-2707.91	0
terms used	8	5
train RMSE	0.00	8.90
test RMSE	30.31	5.22

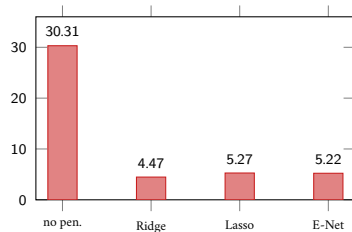
The middle road

Three powers are deleted outright (like Lasso) and the surviving odd powers u, u^3, u^5 — the correlated group — are all kept with modest, same-sign weights (like Ridge). Re-run the survey and this set barely changes.

All Four Models



RMSE on the 8 unseen houses



Model	terms	train	test
no penalty	8	0.00	30.31
Ridge $\lambda=1$	8	7.36	4.47
Lasso $\lambda=1$	1	6.80	5.27
E-Net $\lambda=1, \rho=0.8$	5	8.90	5.22

The model with the **best** training score is the **worst** on new houses. Every penalty cut the real error by about **6x**.

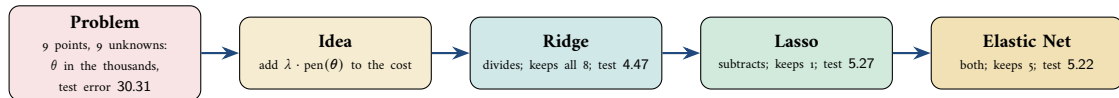
Ridge vs Lasso vs Elastic Net — Reference Card

	Ridge (L_2)	Lasso (L_1)	Elastic Net
Penalty	$\lambda \sum \theta_j^2$	$\lambda \sum \theta_j $	$\lambda [\rho \sum \theta_j + (1 - \rho) \sum \theta_j^2]$
Reaches exactly 0	never	yes	yes
Feature selection	no	yes	yes
Closed form	yes	no	no
Solver	linear algebra	coordinate descent	coordinate descent
Correlated columns	shares between them	picks one at random	keeps the group
Knobs	λ	λ	λ and ρ
scikit-learn	Ridge(alpha)	Lasso(alpha)	ElasticNet(alpha, l1_ratio)
On our 9 houses	8 terms, test 4.47	1 term, test 5.27	5 terms, test 5.22
Choose it when	every column helps a bit	most columns are useless	correlated groups <i>and</i> junk

A decision rule you can actually use

Start with **Ridge** — it is the safest and has a closed form. Move to **Lasso** when you must *explain* the model or you suspect most columns are noise. Move to **Elastic Net** when Lasso's chosen list keeps changing every time you re-run it.

Summary — Regularisation



- With as many unknowns as data points a model can fit perfectly and still be useless. The visible symptom is coefficients in the thousands with alternating signs.
- **Ridge** $\lambda \sum \theta_j^2$: closed form $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$, always invertible, **divides** each coefficient — shrinks but never deletes.
- **Lasso** $\lambda \sum |\theta_j|$: no closed form, weak terms hit **exactly zero**, so it selects features.
- **Elastic Net** mixes both: deletes junk *and* keeps correlated groups together, giving a stable list.

Appendix: Implementation

A1. The Nine Houses and the Eight Powers

Running the code in this appendix reproduces **every number** on the Regularisation slides.

```

1 import numpy as np
2 from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
3
4 # the nine houses we are allowed to learn from
5 x = np.array([0.8, 1.0, 1.2, 1.4, 1.6, 1.8, 2.0, 2.2, 2.4])      # k sq.ft
6 y = np.array([70., 69., 67., 87., 74., 97., 102., 113., 112.])  # lakh
7
8 # eight unseen houses, at the areas in between
9 xt = np.array([0.9, 1.1, 1.3, 1.5, 1.7, 1.9, 2.1, 2.3])
10 yt = np.round(30 + 34*xt + np.random.default_rng(1227).normal(0, 6, 8), 1)
11
12 u, ut = (x - 1.6)/0.8, (xt - 1.6)/0.8      # rescale area to about [-1, 1]
13 P = np.column_stack([u**k for k in range(1, 9)])  # u, u^2, ..., u^8
14 Pt = np.column_stack([ut**k for k in range(1, 9)])
15
16 rmse = lambda m, A, b: float(np.sqrt(np.mean((m.predict(A) - b)**2)))
  
```

The only trick in the whole section: **P** turns one column into eight, and everything after that is ordinary linear regression.

A2. The Four Fits, and What They Print

```

1 def report(tag, model):
2     model.fit(P, y)
3     print(f"{tag:18s} "
4           f"train={rmse(model,P,y):5.2f} "
5           f"test={rmse(model,Pt,yt):6.2f} "
6           f"terms={int((abs(model.coef_)>1e-8).sum())}")
7     print("    ", np.round(model.coef_, 2))
8
9 report("no penalty", LinearRegression())
10 report("Ridge lam=1", Ridge(alpha=1))
11 report("Lasso lam=1", Lasso(alpha=1, max_iter=500000))
12 report("ElasticNet",
13        ElasticNet(alpha=1, l1_ratio=0.8, max_iter=500000))

```

```

no penalty  train= 0.00 test= 30.31 terms=8
[  10.10  433.62  181.16 -2626.76
 -378.31 4918.04  208.05 -2707.91]

Ridge lam=1  train= 7.36 test=  4.47 terms=8
[16.68  2.53  5.34  1.35
  1.12  0.74 -0.75  0.35]

Lasso lam=1  train= 6.80 test=  5.27 terms=1
[22.93  0.    0.    0.
  0.    0.    0.    0. ]

ElasticNet   train= 8.90 test=  5.22 terms=5
[12.97  0.44  4.54  0.02
  1.37  0.    0.    0. ]

```

Read the test column: 30.31 $\rightarrow$ 4.47 $\rightarrow$ 5.27 $\rightarrow$ 5.22. Read the terms column: 8 $\rightarrow$ 8 $\rightarrow$ 1 $\rightarrow$ 5.

A3. Ridge and Lasso by Hand, in Four Lines

Because $u = -1, -0.75, \dots, 1$ is symmetric, the columns u and u^2 are orthogonal and $\mathbf{X}^T \mathbf{X}$ is diagonal — so each coefficient can be solved on its own.

```

1 A = np.column_stack([u, u**2])
2 c = A.T @ (y - y.mean())      #  $X'y \rightarrow [95.00, 8.54]$ 
3 d = np.diag(A.T @ A)         #  $X'X$  diagonal  $\rightarrow [3.750, 2.766]$ 
4
5 for lam in [0, 1, 3, 10, 30]:  # RIDGE: divide
6     print(lam, np.round(c / (d + lam), 2))
7
8 for lam in [0, 10, 20, 40]:    # LASSO: subtract
9     soft = np.sign(c) * np.maximum(np.abs(c) - lam/2, 0)
10    print(lam, np.round(soft / d, 2))

```

RIDGE	LASSO
0 [25.33 3.09]	0 [25.33 3.09]
1 [20.00 2.27]	10 [24.00 1.28]
3 [14.07 1.48]	20 [22.67 0.00]
10 [6.91 0.67]	40 [20.00 0.00]
30 [2.81 0.26]	

The one-line difference

$c/(d+lam)$ can never reach zero.
 $\text{maximum}(|c|-lam/2, 0)$ reaches zero the moment the penalty exceeds the evidence.

A4. The Bias–Variance Experiment

```

1 TRUE, SIG, REPS = lambda a: 30 + 34*a, 6.0, 500
2 grid = np.linspace(0.8, 2.4, 17); gu = (grid - 1.6)/0.8
3 design = lambda uu, d: np.column_stack([np.ones_like(uu)] +
4                                         [uu**k for k in range(1, d+1)])
5
6 def experiment(degree, lam=0.0):
7     r = np.random.default_rng(99)
8     A, Ag = design(u, degree), design(gu, degree)
9     D = np.eye(degree+1); D[0, 0] = 0      # never penalise the intercept
10    curves = np.zeros((REPS, len(grid)))
11    for s in range(REPS):
12        yy = TRUE(x) + r.normal(0, SIG, 9)    # a FRESH survey every time
13        c = np.linalg.solve(A.T@A + max(lam, 1e-12)*D, A.T@yy)
14        curves[s] = Ag @ c
15    bias2 = np.mean((curves.mean(0) - TRUE(grid))**2)    # where the cloud sits
16    var = np.mean(curves.var(0))    # how wide it is
17    return round(bias2, 2), round(var, 2), SIG**2, round(bias2+var+SIG**2, 2)
18
19 for deg in [1, 2, 4, 8]: print("degree", deg, experiment(deg))
20 for lam in [0, 0.01, 0.1, 0.3, 1, 3, 10]:
21     print("degree 8, lam =", lam, experiment(8, lam))

```

Only this loop can separate bias from variance — and only because we *know* the truth. On real data you get one survey, one curve, one number.

A5. How You Should Actually Write It: a Pipeline

```

1 from sklearn.pipeline      import make_pipeline
2 from sklearn.preprocessing import PolynomialFeatures, StandardScaler
3 from sklearn.linear_model  import RidgeCV, LassoCV, ElasticNetCV
4 from sklearn.model_selection import cross_val_score
5
6 alphas = np.logspace(-3, 2, 30)
7 X, Xt  = x.reshape(-1, 1), xt.reshape(-1, 1)      # raw area, nothing else
8
9 models = {
10     "ridge": make_pipeline(PolynomialFeatures(8), StandardScaler(),
11                           RidgeCV(alphas=alphas)),
12     "lasso": make_pipeline(PolynomialFeatures(8), StandardScaler(),
13                           LassoCV(alphas=alphas, cv=3, max_iter=500000)),
14     "enet" : make_pipeline(PolynomialFeatures(8), StandardScaler(),
15                           ElasticNetCV(alphas=alphas, cv=3, max_iter=500000,
16                                         l1_ratio=[.1,.5,.7,.9,.95,.99,1])),
17 }
18 for name, pipe in models.items():
19     pipe.fit(X, y)      # the powers AND the scaling live inside
20     print(f"{name:6s} alpha*={pipe[-1].alpha_:8.4f} test={rmse(pipe,Xt,yt):6.2f}")

```

Why a Pipeline and not three separate steps

The powers and the scaler are rebuilt **inside every cross-validation fold**. Written any other way, the held-out rows help compute μ and σ , the reported score is optimistic, and the deployed model disappoints.